

电商 SingleFile 页面 AI 相关评论 / 问大家提取工具开发说明书 v1.0

1. 项目目标

请用 Python 开发一个本地脚本，用于批量处理已经通过 Chrome 插件 SingleFile 保存下来的电商商品页面 HTML 文件。

本项目第一阶段只做本地处理，不调用任何大模型 API，不访问网络，不打开浏览器，不请求淘宝、京东、拼多多、抖音等网站。

核心目标：

1. 批量读取本地 `.html` / `.htm` 文件。
2. 从页面中提取普通商品评价。
3. 从淘宝页面中提取“问大家”的问题与回答。
4. 从评论与问答中筛选出可能与 AI 信息来源有关的内容。
5. 输出 Excel，方便后续人工复核与统计。

第一阶段重点识别以下内容：

- 消费者是否提到 AI、豆包、DeepSeek、ChatGPT、Kimi、通义千问、文心一言、腾讯元宝等。
- 消费者是否表达“问 AI / 让 AI 推荐 / AI 对比 / AI 建议 / AI 说可以买”等购买决策路径。
- 淘宝“问大家”中是否出现与 AI 信息来源有关的问题或回答。

第一阶段暂不做大模型语义判断。所有命中均基于本地关键词与规则。

2. 重要原则

2.1 本项目不是爬虫

页面已经由用户提前保存到本地。

脚本不得做以下事情：

- 不访问互联网
- 不发送 HTTP 请求
- 不登录淘宝
- 不调用浏览器
- 不绕过反爬
- 不调用大模型 API

脚本只处理本地 HTML 文件。

2.2 不把整页 HTML 交给大模型

SingleFile 页面体积很大，包含图片、CSS、JS、页面状态等大量噪音。

第一阶段处理流程：

本地 HTML 文件
→ 解析页面
→ 提取评论 / 问大家
→ 清洗文本
→ 关键词筛选
→ 输出 Excel

2.3 第一阶段只找“可能相关”

第一阶段的任务是找出候选内容，不做最终归因。

输出口径应为：

可能与 AI 信息来源有关的消费者评论 / 问大家内容

不要直接输出：

这些评论已经被 AI-GEO 影响

3. 输入与输出

3.1 输入

输入为一个本地文件夹，包含多个 SingleFile 保存的 HTML 文件。

支持：

- `.html`
- `.htm`
- 中文路径
- 中文文件名
- 子文件夹递归读取
- 大体积 HTML 文件
- 单个文件失败时继续处理其他文件

命令示例：

```
python main.py --input "./html_pages" --output "./output/ai_related_reviews.xlsx" --keywords "./config/ai_keywords.yaml" --recursive
```

3.2 输出

输出一个 Excel 文件，包含以下 Sheet：

1. `ai_candidates`：可能与 AI 有关的评论 / 问大家内容，核心结果表。
2. `content_all`：所有成功提取的评论、问题、回答。
3. `qa_pairs_raw`：淘宝“问大家”原始结构化表。
4. `reviews_raw`：普通商品评价原始结构化表。
5. `summary_by_product`：按商品汇总。
6. `summary_by_keyword`：按 AI 关键词汇总。
7. `errors`：错误与失败记录。
8. `debug_samples`：低置信度或无法稳定配对的文本样本，便于修正解析规则。

4. 目录结构建议

```
ecommerce_ai_comment_extractor/  
  main.py  
  requirements.txt  
  README.md  
  
  config/  
    ai_keywords.yaml  
  
  extractor/  
    __init__.py  
    platform_detect.py  
    html_loader.py  
    product_meta.py  
    taobao_qa.py  
    reviews.py  
    generic_blocks.py  
  
  analysis/  
    ai_keyword_matcher.py  
    evidence_rules.py  
    summary.py  
  
  utils/  
    text_cleaner.py  
    dedupe.py  
    excel_writer.py  
    logger.py  
  
  output/
```

5. requirements.txt

```
beautifulsoup4  
lxml  
pandas  
openpyxl  
pyyaml  
tqdm
```

可选：

```
selectolax
```

第一版优先使用 BeautifulSoup + lxml，方便调试。

6. 命令行参数

需要支持：

```
python main.py  
--input "./html_pages"  
--output "./output/ai_related_reviews.xlsx"  
--keywords "./config/ai_keywords.yaml"  
--recursive
```

参数说明：

--input	输入文件夹，必填
--output	输出 Excel 文件路径，必填
--keywords	AI 关键词配置文件，默认 ./config/ai_keywords.yaml
--recursive	是否递归读取子文件夹，默认 False
--debug	是否输出 debug 样本，默认 True
--max-files	最多处理多少个文件，调试用，可选

7. 需要处理的数据对象

第一阶段需要处理三类文本：

1. 普通商品评价：review
2. 淘宝“问大家”的问题：qa_question
3. 淘宝“问大家”的回答：qa_answer

最终统一进入 `content_all` 表。

8. 商品基础信息字段

尽量从页面中提取商品信息。

字段：

```
source_file
platform
product_title
shop_name
product_url
product_id
page_title
```

说明：

- `source_file` 必须有。
- `platform` 必须有，无法识别则为 `unknown`。
- 其他字段提取不到可以留空。
- 不要因为商品信息不完整而中断评论提取。

9. 平台识别

实现函数：

```
def detect_platform(html_text: str, file_path: str) -> str:
    pass
```

返回值：

```
taobao
tmall
jd
pdd
douyin
unknown
```

淘宝 / 天猫识别参考词：

```
taobao.com
tmall.com
淘宝
天猫
```

问大家
已买到的宝贝
购物车
收藏夹

如果页面中出现“问大家”，优先按淘宝 / 天猫处理。

10. 淘宝“问大家”提取

10.1 目标

提取类似以下结构：

问大家 · 1605

问题：这种眼灯是真有用吗，还是智商税，这么贵！

回答：有用，这钱花的值

问题：air和pro和max推荐哪个呢？谢谢

回答：根据面积来就行了

问题：灯光柔和不刺眼，不晃眼吗

回答：好光可以调

10.2 输出 Sheet：qa_pairs_raw

字段：

```
source_file
platform
product_title
shop_name
product_url
product_id
qa_order
question_text_raw
question_text_clean
answer_text_raw
answer_text_clean
answer_user_status
answer_count
question_tags
is_buyer_answer
qa_hash
extract_confidence
```

10.3 问大家标签提取

页面顶部可能出现：

全部 1605
质高 74
好用 49
灯光 33
效果 33
刺眼 29
眼睛 18

需要提取为：

question_tags = 全部:1605;质高:74;好用:49;灯光:33;效果:33;刺眼:29;眼睛:18

如果只提取到标签名，也可以保存为：

质高;好用;灯光;效果;刺眼;眼睛

10.4 问答配对规则

优先寻找页面 DOM 中重复出现的问答卡片。

每个卡片中通常包含：

- 问题文本
- 回答文本
- “已购”标记
- “查看全部回答”
- 更多按钮

清洗时去掉：

查看全部回答
查看全部
展开
收起
更多
...

保留消费者原话，例如：

智商税
这么贵

推荐哪个
刺眼
不晃眼
写作业
手影

10.5 低置信度处理

如果问题和回答无法稳定配对，不要丢弃。

保存到 `debug_samples`：

```
source_file
platform
debug_type = qa_pair_uncertain
raw_block_text
possible_question
possible_answer
reason
```

11. 普通商品评价提取

11.1 输出 Sheet: reviews_raw

字段：

```
source_file
platform
product_title
shop_name
product_url
product_id
review_order
review_text_raw
review_text_clean
review_time
rating
sku
user_name_masked
is_followup
followup_text
merchant_reply
image_count
video_count
like_count
```



```
review_hash
extract_confidence
```

11.2 评论提取原则

优先提取消费者正文。

需要区分：

- 消费者评论
- 追评
- 商家回复
- 平台标签
- 商品参数
- 店铺广告
- 推荐商品文案

`merchant_reply` 可以单独保存，但不要混入 `review_text_clean`。

提取不到评论时返回空列表，不中断程序。

12. 统一内容表 `content_all`

评论、问大家问题、问大家回答统一进入 `content_all`。

字段：

```
content_id
record_type
source_file
platform
product_title
shop_name
product_url
product_id
content_role
content_text_raw
content_text_clean
parent_text_clean
sku
user_name_masked
user_status
content_order
content_hash

ai_related
ai_source_hit
ai_source_terms
```

```
ai_action_hit
ai_action_terms
purchase_context_hit
purchase_context_terms
decision_context_hit
decision_context_terms

matched_keywords
matched_sentence
match_score
evidence_level
evidence_reason
extract_confidence
```

字段说明：

```
record_type:
  review
  qa

content_role:
  review
  question
  answer

parent_text_clean:
  如果 content_role = answer，则保存对应问题
  其他情况可为空
```

13. AI 关键词配置

关键词放在：

```
config/ai_keywords.yaml
```

13.1 第一阶段关键词配置示例

```
ai_sources:
  strong:
    - 豆包
    - DeepSeek
    - deepseek
    - ChatGPT
    - chatgpt
    - GPT
```

- Kimi
- kimi
- 通义千问
- 通义
- Qwen
- 文心一言
- 百度AI
- 腾讯元宝
- 元宝
- Gemini
- Copilot

generic:

- AI
- A.I.
- ai助手
- AI助手
- 人工智能
- 智能助手
- 智能问答

ai_actions:

- 推荐
- 建议
- 问了
- 问一下
- 问了下
- 问AI
- 让AI
- 用AI
- AI说
- AI推荐
- AI建议
- 查了
- 搜了
- 对比
- 比较
- 分析
- 告诉我
- 种草
- 让我买
- 帮我选
- 选哪个
- 推荐哪个

purchase_context:

- 买
- 购买
- 入手
- 下单
- 选了

- 最后选
- 冲了
- 到手
- 收到
- 退货
- 换货
- 推荐买吗
- 值得买吗
- 值不值

decision_context:

- 推荐哪个
- 选哪个
- 哪个好
- 有没有用
- 有用吗
- 智商税
- 值不值
- 这么贵
- 对比
- 比较
- 怎么选
- 适合吗
- 后悔吗

14. 重要：AI 关键词匹配规则

14.1 不要简单匹配小写 ai

页面中可能出现商品型号，例如：

Air
air和pro和max推荐哪个

如果简单匹配小写 `ai`，会把 `air` 错判为 AI。

因此必须实现安全匹配规则。

14.2 AI 匹配规则

`AI` 可匹配：

AI推荐
问AI
让AI
AI说
AI助手

AI智能
用AI查

`ai` 小写只在以下情况匹配：

ai助手
ai推荐
问ai
用ai
让ai

不要把以下内容匹配为 AI：

air
Air
AIR
chair
pair
rain

14.3 推荐实现

对 `AI` / `ai` 这种短词，使用正则边界或上下文规则。

示例思路：

```
AI_PATTERN = re.compile(  
    r'(?<![A-Za-z])(?:AI|A\.I\.|ai)(?![A-Za-z])(?:问|用|让|找|请|打开|咨询)(?:AI|ai)(?:推荐|建议|助手|  
    分析|对比|说)',  
    re.IGNORECASE  
)
```

但需要注意：

- `re.IGNORECASE` 可能导致 `Air` 被误伤。
- 更稳的方式是对 `AI` 单独做规则，不对普通英文单词做无脑包含匹配。
- 对 `DeepSeek` / `ChatGPT` / `Kimi` / `Qwen` 可以做大小写不敏感匹配。
- 对 `ai` 单独要求前后不能是英文字母。

15. AI 候选判断逻辑

实现函数：

```
def analyze_ai_related(record: dict, keywords: dict) -> dict:  
    pass
```

对每条 `content_all` 记录进行分析。

15.1 命中字段

```
ai_related  
ai_source_hit  
ai_source_terms  
ai_action_hit  
ai_action_terms  
purchase_context_hit  
purchase_context_terms  
decision_context_hit  
decision_context_terms  
matched_keywords  
matched_sentence  
match_score  
evidence_level  
evidence_reason
```

15.2 候选判定

进入 `ai_candidates` 的条件：

满足以下任一条件：

条件 1：命中明确 AI 来源词

例如：

```
豆包  
DeepSeek  
ChatGPT  
Kimi  
通义千问  
文心一言  
腾讯元宝  
AI助手  
人工智能
```

条件 2：命中安全 AI 词 + 行为词

例如：

问AI推荐
让AI帮我选
AI说这个可以买
AI对比了几款

条件 3：问大家中出现“推荐 / 选哪个 / 怎么选 / 值不值”等决策语境，同时父问题或回答命中 AI 来源词

例如：

问题：问了豆包，air和pro推荐哪个？
回答：根据面积来就行。

16. 证据等级

第一阶段证据等级只围绕“AI 相关可能性”判断。

A 级：明确 AI 决策路径

满足：

AI 来源词 + AI 行为词 + 购买 / 推荐 / 对比 / 选择语境

示例：

问了豆包推荐这款，买回来确实不错。
DeepSeek 对比了几款，最后选了这个。
ChatGPT 说这个适合孩子写作业。
让AI帮我选，推荐了这款。

输出：

evidence_level = A
evidence_reason = 明确提到AI来源，并与推荐、对比、选择或购买有关

B 级：提到 AI 来源，但购买影响较弱

满足：

AI 来源词出现，但没有明确推荐、选择、购买动作

示例：

之前在豆包里看到过。
AI里也有人说这个牌子。

输出：

evidence_level = B
evidence_reason = 提到AI来源，但没有明确购买决策动作

C 级：疑似 AI 语境，需要人工复核

满足：

出现智能助手、智能问答、推荐、分析、对比等词，但 AI 来源不明确

示例：

智能助手推荐的，不知道准不准。
看了几个对比之后买的。

输出：

evidence_level = C
evidence_reason = 存在疑似AI或决策语境，需要人工复核

D 级：非 AI 相关

默认：

evidence_level = D
evidence_reason = 未命中AI来源或疑似AI语境

注意：

- D 级不进入 ai_candidates。
- content_all 中保留所有记录，便于回查。

17. match_score 设计

给每条内容一个简单分数，便于排序。

建议规则：

命中强 AI 来源词：+5
命中通用 AI 来源词：+3
命中 AI 行为词：+3
命中购买语境：+2
命中决策语境：+2
AI 来源词与行为词在同一句：+3
AI 来源词与购买语境在同一句：+2
命中内容来自“问大家问题”：+1
命中内容来自“问大家回答”：+1

分数越高，越值得人工优先复核。

18. matched_sentence 提取

实现函数：

```
def extract_matched_sentence(text: str, matched_terms: list[str]) -> str:  
    pass
```

规则：

1. 按中文标点切句：
2. 。
3. !
4. ?
5. ;
6. ；
7. 换行
8. 返回包含命中关键词的句子。
9. 如果多个句子命中，用 `|||` 拼接。
10. 如果文本很短，直接返回全文。
11. 如果无法切句，返回关键词前后各 40 个字符。

19. 文本清洗规则

实现函数：

```
def clean_text(text: str) -> str:  
    pass
```

清洗规则：

1. 去掉首尾空格。
2. 合并多个空格。
3. 合并多个换行。
4. 去掉平台操作文案：
5. 查看全部回答
6. 查看全部
7. 展开
8. 收起
9. 查看更多
10. 更多
11. 追评
12. 默认好评
13. 去掉孤立的省略号按钮文本：
14. ...
15. ...
16. 保留消费者原话中的情绪词：
17. 智商税
18. 太贵
19. 不值
20. 真香
21. 后悔
22. 保留品牌词、AI 词、型号词。
23. 同时保存 raw_text 与 clean_text。

20. 去重规则

实现函数：

```
def dedupe_records(records: list[dict]) -> list[dict]:  
    pass
```

20.1 content_hash

对所有记录生成：

```
content_hash = md5(platform + product_id + content_role + content_text_clean +
parent_text_clean)
```

如果 `product_id` 为空：

```
content_hash = md5(source_file + content_role + content_text_clean + parent_text_clean)
```

20.2 QA 去重

问大家中问题与回答配对后：

```
qa_hash = md5(platform + product_id + question_text_clean + answer_text_clean)
```

20.3 评论去重

普通评论：

```
review_hash = md5(platform + product_id + review_text_clean + review_time + sku)
```

去重后保留第一次出现的记录。

21. Excel 输出字段

21.1 ai_candidates

这是第一阶段核心 Sheet。

字段：

```
content_id
source_file
platform
product_title
shop_name
product_url
product_id
record_type
content_role
content_text_clean
parent_text_clean
user_status
sku
ai_source_terms
```

```
ai_action_terms
purchase_context_terms
decision_context_terms
matched_keywords
matched_sentence
match_score
evidence_level
evidence_reason
content_hash
```

排序规则：

```
先按 evidence_level: A → B → C
再按 match_score 降序
再按 source_file
```

21.2 content_all

字段：

```
content_id
record_type
source_file
platform
product_title
shop_name
product_url
product_id
content_role
content_text_raw
content_text_clean
parent_text_clean
sku
user_name_masked
user_status
content_order
ai_related
ai_source_hit
ai_source_terms
ai_action_hit
ai_action_terms
purchase_context_hit
purchase_context_terms
decision_context_hit
decision_context_terms
matched_keywords
matched_sentence
match_score
```

```
evidence_level  
evidence_reason  
extract_confidence  
content_hash
```

21.3 qa_pairs_raw

字段:

```
source_file  
platform  
product_title  
shop_name  
product_url  
product_id  
qa_order  
question_text_raw  
question_text_clean  
answer_text_raw  
answer_text_clean  
answer_user_status  
answer_count  
question_tags  
is_buyer_answer  
qa_hash  
extract_confidence
```

21.4 reviews_raw

字段:

```
source_file  
platform  
product_title  
shop_name  
product_url  
product_id  
review_order  
review_text_raw  
review_text_clean  
review_time  
rating  
sku  
user_name_masked  
is_followup  
followup_text  
merchant_reply  
image_count
```

```
video_count
like_count
review_hash
extract_confidence
```

21.5 summary_by_product

字段:

```
platform
product_title
shop_name
product_id
source_file_count
review_count
qa_question_count
qa_answer_count
content_count
ai_candidate_count
a_level_count
b_level_count
c_level_count
ai_candidate_rate
top_ai_source_terms
top_ai_action_terms
top_examples
```

21.6 summary_by_keyword

字段:

```
keyword_type
keyword
hit_count
example_1
example_2
example_3
```

`keyword_type` 包括:

```
ai_source
ai_action
purchase_context
decision_context
```

21.7 errors

字段：

```
source_file
platform
error_stage
error_type
error_message
traceback
```

21.8 debug_samples

字段：

```
source_file
platform
debug_type
raw_block_text
possible_question
possible_answer
reason
```

22. 日志要求

运行时打印并保存日志到：

```
./output/run.log
```

日志包含：

```
开始处理文件数量
当前处理文件名
平台识别结果
商品标题
提取评论数量
提取问大家数量
content_all 数量
AI候选数量
A/B/C等级数量
错误文件数量
输出文件路径
```

23. 主要函数要求

至少实现以下函数：

```
def load_html(file_path: str) -> str:
    """读取本地 HTML 文件，支持中文路径和常见编码。"""
    pass

def iter_html_files(input_dir: str, recursive: bool = False) -> list[str]:
    """遍历输入文件夹中的 html/htm 文件。"""
    pass

def detect_platform(html_text: str, file_path: str) -> str:
    """识别平台。"""
    pass

def extract_product_meta(html_text: str, soup, source_file: str, platform: str) -> dict:
    """提取商品基础信息。"""
    pass

def parse_taobao_qa(html_text: str, soup, meta: dict) -> list[dict]:
    """提取淘宝问大家的问题和回答。"""
    pass

def parse_reviews(html_text: str, soup, meta: dict) -> list[dict]:
    """提取普通商品评价。"""
    pass

def clean_text(text: str) -> str:
    """清洗文本，但保留消费者原话。"""
    pass

def build_content_all(reviews: list[dict], qa_pairs: list[dict]) -> list[dict]:
    """将评论、问题、回答合并为统一内容表。"""
    pass

def load_keywords(keyword_file: str) -> dict:
    """读取 YAML 关键词配置。"""
    pass

def safe_match_ai_sources(text: str, keywords: dict) -> list[str]:
    """安全匹配 AI 来源词，避免把 Air / air 误判为 AI。"""
    pass

def analyze_ai_related(record: dict, keywords: dict) -> dict:
    """分析一条内容是否可能与 AI 信息来源有关。"""
    pass

def extract_matched_sentence(text: str, matched_terms: list[str]) -> str:
    """提取命中关键词所在句子。"""
```



```
pass

def assign_evidence_level(record: dict) -> dict:
    """根据命中情况分配 A/B/C/D 证据等级。"""
    pass

def dedupe_records(records: list[dict]) -> list[dict]:
    """根据 hash 去重。"""
    pass

def build_summaries(content_all: list[dict], ai_candidates: list[dict]) -> dict:
    """生成汇总表。"""
    pass

def write_excel(output_path: str, sheets: dict) -> None:
    """输出多 Sheet Excel。"""
    pass
```

24. 开发优先级

第一优先级：必须完成

1. 批量读取本地 HTML。
2. 识别淘宝 / 天猫页面。
3. 提取商品标题。
4. 提取淘宝“问大家”的问题与回答。
5. 提取普通商品评价，如果页面中存在。
6. 合并为 `content_all`。
7. 用本地关键词识别 AI 相关候选。
8. 特别处理 `AI` 与 `Air/air` 的误判问题。
9. 输出 Excel。
10. 保存错误日志。

第二优先级：增强稳定性

1. 更多平台字段提取。
2. 更好的问答配对。
3. 普通评论时间、SKU、评分提取。
4. debug 样本输出。
5. 提取置信度评分。

第三优先级：后续再做

1. 京东、拼多多、抖音电商专用解析。
2. GEO 卖点词识别。
3. 大模型复核模块。
4. 自动生成分析报告。

25. README 示例

README 中需要包含：

```
pip install -r requirements.txt

python main.py
  --input "./html_pages"
  --output "./output/ai_related_reviews.xlsx"
  --keywords "./config/ai_keywords.yaml"
  --recursive
```

运行后检查：

```
output/ai_related_reviews.xlsx
output/run.log
```

优先查看：

```
ai_candidates
content_all
qa_pairs_raw
errors
debug_samples
```

26. 验收标准

第一阶段完成后，需要满足：

1. 脚本可以在本地运行，不访问网络。
2. 可以批量处理一个文件夹中的 SingleFile HTML。
3. 淘宝“问大家”可以被提取为问题和回答。
4. 普通评论如果存在，也能进入 reviews_raw。
5. 所有评论、问题、回答都进入 content_all。
6. 命中 AI 相关关键词的内容进入 ai_candidates。
7. 不把 `air` 和 `pro` 和 `max` 推荐哪个 误判为 AI。
8. Excel 中可以看到原文、命中关键词、命中句子、证据等级、来源文件。
9. 单个文件失败不影响整体运行。
10. 有 errors 和 run.log 方便排错。

27. 第一阶段最终产物

最终交付：

```
main.py
requirements.txt
README.md
config/ai_keywords.yaml
extractor/
analysis/
utils/
```

运行后生成：

```
output/ai_related_reviews.xlsx
output/run.log
```

Excel 中最重要的 Sheet 是：

```
ai_candidates
```

它应该回答：

哪些消费者评论 / 问大家内容，可能明确提到了 AI、豆包、DeepSeek、ChatGPT 等信息来源？
这些内容来自哪个商品、哪个页面、哪个问题或哪条评论？
它们为什么被判定为 AI 相关候选？